

Motion Planing of Powered-Caster Vehicle Based on Gaussian Process

Xiao Wang²[0009-0002-0162-4301], Fei Zhao^{1,2}[0000-0002-1405-5593], Heyuan Li^{1,2}[0009-0001-4327-6301], Baolin Liu^{1,2}[0009-0006-3303-6810], Chenwei Gong^{1,2}[0000-0001-7372-7612], and Xuesong Mei^{1,2}[0000-0001-6505-2774]

¹ School of Mechanical Engineering, Xi'an Jiaotong University, Xi'an 710049, China
ztzhao@mail.xjtu.edu.cn

² Shaanxi Key Laboratory of Intelligent Robots and School of Mechanical Engineering, Xi'an Jiaotong University, Xi'an 710049, Shaanxi, China

Abstract. This paper presents a motion planning strategy for powered-caster omnidirectional robots within the ROS framework that is driven by Gaussian Processes and integrates cubic spline interpolation to enhance both navigation precision and motion smoothness. By dynamically adjusting the Gaussian kernel based on trajectory costs and collision risks, the algorithm achieves superior positional tracking accuracy and computational efficiency, operating at up to 1kHz. Experimental validation on the robot demonstrates improved performance over conventional techniques, especially in accuracy and computational speed.

Keywords: ROS · Gaussian Process · Motion Planing · Powered-Caste.

1 Introduction

Motion planning is crucial in robotics as it enables robots to autonomously navigate and perform tasks efficiently and safely in dynamic environments[1]. Robot has various applications in daily life and Industry, such as autonomous vacuum cleaners navigating efficiently through household environments, robotic arms precisely assembling components in manufacturing plants, and service robots safely interacting with humans in public spaces.

Fox [3] introduced the Dynamic Window Approach (DWA) in 1996, a technique tailored for differential-drive robots that employs dynamic windowing to respect kinematic constraints. Its integration into ROS led to prevalent use in commercial robotics. Meanwhile, Quinlan's Timed Elastic Band (TEB)[4] was later incorporated into ROS[8] by Rösmann[5], also advancing the field with refined path planning capabilities. Subsequently, a myriad of refined algorithms based on DWA and TEB have been extensively adopted in autonomous vehicles and service robots. For instance, Zai[6] integrated TEB with the A* algorithm to facilitate navigation for Ackermann steering geometry-based robots. Dobrevski[7] proposed the Adaptive Dynamic Window Approach (ADWA), an advancement that enhances a robot's capability to evade dynamic obstacles, thereby improving its performance in dynamic environments. Zheng[9] employed reinforcement

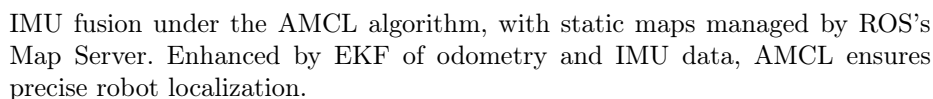
learning to dynamically adjust TEB parameters in real-time, thereby enhancing the algorithm’s capability to avoid dynamic obstacles effectively. Jiang[10] proposed a reinforcement learning-based variant of the DWA algorithm, specifically tailored for operation in scenarios characterized by dense obstacles. Currently, the majority of motion planning algorithms are primarily integrated with differential drive models and Ackermann steering models, with scant few algorithms directly designed to leverage the characteristics of omnidirectional mobile robots. Imamoglu[11] conducted a comparative analysis of the TEB and DWA algorithms when applied to omnidirectional mobile robots, revealing that TEB yields significantly smoother output trajectories compared to DWA. However, TEB’s performance is mitigated by an abundance of parameters and constraints, resulting in a lower computational frequency. Moreover, this excess complexity can inadvertently lead to planning maneuvers reminiscent of car-like turning patterns for the omnidirectional robot, an inefficiency not inherent to its full mobility capabilities.

Liu[12] proposed a trajectory planning method for 7-DOF manipulator using Gaussian Processes combined with a time optimizer, which generates collision-free trajectories through a sampling approach, offering excellent real-time performance and stability. For the first time, this paper adapts and applies this methodology to an omnidirectional mobile robot platform, further extending its application scope and exploring its effectiveness in a different robotic domain. This adaptation involves addressing unique challenges presented by the maneuverability and dynamic behavior of omnidirectional robots, potentially enhancing their autonomous navigation capabilities in complex environments. The kinematics algorithm employed in this paper’s robot utilizes the control approach for omni-directional mobile robots with active wheels, as proposed by Jia[13, 14]. The main contributions are summarized as follows:

1. A software system for a load-carrying robot was designed based on ROS, encompassing localization, navigation, vision, and control systems, and subsequently deployed on an actual robotic platform.
2. A load-carrying robot was designed, and the active wheel kinematics algorithm was implemented on the robot constructed in this paper.
3. An improvement to the Gaussian Process-based motion planning algorithm was made based on the ROS system and applied to the load-carrying robot designed in this paper.

2 Robot System Framework Design Based on ROS

This chapter outlines a holistic robot control system based on ROS. As shown in Fig.1, integrating key functionalities such as navigation, vision, motion control, and simulation. The system architecture consists of localization, planning, control, and vision subsystems. The positioning system, informed by sensor data and control feedback, determines the robot’s location within a map, informing the planning system which then makes decisions based on combined localization and vision data. Localization accuracy is achieved through LiDAR, odometry, and



The control aspect relies on ROS Control for actuator command conversion and odometry calculation. Real-world robot control utilizes EtherCAT for motor communication, while Gazebo simulation emulates this process for virtual testing. Planning involves Gaussian Process optimized trajectories, refined with cubic splines, controlled by a PID loop ensuring trajectory adherence. Vision processing with YOLOv5 detects targets, followed by ICP for cylinder fitting to calculate the workpiece’s relative position to the robot, directing motion accordingly.

3 Kinematics of Powered-Caster Holonomic Robot

This section adopts a scheme with biased Powered-Caster to achieve omnidirectional movement of the robot. This chapter will introduce the forward and inverse kinematics of the active caster omnidirectional mobile robot, as well as the calculation of odometry. The implementation of this part of the content is realized in `roscontrol`. As shown in Fig. 2, the origin of the coordinate axis is set at the origin of the robot coordinate system, denoted as C . A represents the projection of the steering axle of the caster wheel onto the coordinate system, while O signifies the projection of the point where the caster wheel touches the ground onto the same coordinate system. The angle α denotes the angular deviation between vector AO and the positive x-axis direction (with a positive value when rotating counterclockwise from the positive x-axis, as depicted in the

diagram). The angle φ represents the angle between vector CA and the x -axis. The length of segment CA is designated as l , and that of segment AO as b .

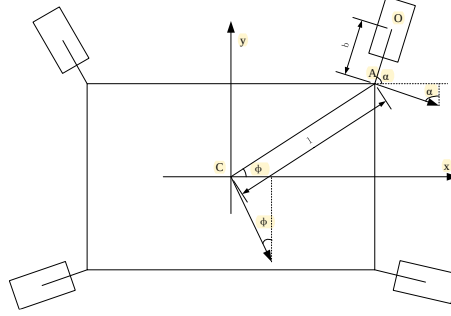


Fig. 2. Powered-Caster Holonomic Robotic Vehicle Geometry Diagram

3.1 Inverse Kinematics

Let $\dot{\theta}$ represent the rotational speed of the driving wheel. We define the positive direction of wheel rotation as "the direction in which the robot moves along the direction of vector AO , corresponding to the wheel's rotation direction at that time." Let $\dot{\alpha}$ represent the rotational speed of the steering actuator, with its positive direction defined as the counterclockwise direction when observed from a top-down perspective on the robot. To analyze the velocity at point C , the origin of the robot's coordinate system, the velocity generated by the driving wheels is as follows:

$$v_o = R\dot{\theta}(i \cos \alpha + j \sin \alpha) \quad (1)$$

Where v_o denotes the velocity of the point of contact between the wheel and the ground, projected onto the robot's coordinate system. R represents the radius of the driving wheel, and i, j are unit vectors along the x and y axes, respectively. The linear velocity generated due to steering by the steering mechanism is as follows:

$$v_a = \dot{\eta}b(\sin \alpha i - \cos \alpha j) \quad (2)$$

Where v_a represents the linear velocity provided to point A by the rotation of the steering mechanism, and $\dot{\eta}$ is the angular velocity of segment AO in the ground coordinate system. Since the rotational speed of the motor does not necessarily equal $\dot{\eta}$ directly, they adhere to the following relationship:

$$\dot{\alpha} + \omega = \dot{\eta} \quad (3)$$

Where ω signifies the angular velocity of the robot's body in the world coordinate system, which is also the angular velocity of segment CA in the world

coordinate system, serving as a given condition in the inverse kinematics equation. $\dot{\alpha}$ represents the angular velocity of the steering mechanism, which is the parameter to be solved for.

$$v_{\omega} = \omega l(\sin \varphi i - \cos \varphi j) \quad (4)$$

Where v_{ω} denotes the velocity vector contributed to point A when the robot's body rotates with an angular velocity of ω in the world coordinate system. Additionally, according to kinetics in theoretical mechanics, the equation for the linear velocity of point C is derived as follows:

$$v_c = v_o + v_a + v_{\omega} \quad (5)$$

By combining equations (1), (2), (4), and (5), we obtain the formulas for linear velocities in the x-direction and y-direction respectively.

$$\begin{aligned} v_{cx} &= R\dot{\theta} \cos \alpha + \dot{\eta}b \sin \alpha + \omega l \sin \varphi \\ v_{cy} &= R\dot{\theta} \sin \alpha - \dot{\eta}b \cos \alpha - \omega l \cos \varphi \end{aligned} \quad (6)$$

Represent $\dot{\eta}$ using v_{cx} :

$$\dot{\eta} = \frac{v_{cx} - (R\dot{\theta} \cos \alpha + \omega l \sin \varphi)}{b \sin \alpha} \quad (7)$$

Substitute this into v_{cy} of Equation (6) and solving for the rotational speed of the driving wheel gives:

$$\dot{\theta} = \frac{1}{R} [v_{cy} \sin \alpha + v_{cx} \cos \alpha + \omega l \sin(\alpha - \varphi)] \quad (8)$$

By combining equations (8),(3) and (7), we get:

$$\dot{\alpha} = \frac{1}{b} (v_{cx} \sin \alpha - v_{cy} \cos \alpha) - \frac{\omega l \cos(\varphi - \alpha)}{b} - \omega \quad (9)$$

With this, the inverse kinematics solution is completed. Each set of steering wheels has distinct parameters, which can be substituted as needed. Next, we proceed to solve the forward kinematics for the omni-directional mobile robot.

3.2 Forward Kinematics

The problem in forward kinematics is as follows: Given the rotational speeds $\dot{\alpha}, \dot{\theta}$ of each wheel, we aim to solve for the robot body's angular velocity v_{cx}, v_{cy}, ω . Utilizing equations (3) and (6), resolving three unknown quantities with just a single set of active wheels is not feasible. However, an omni-directional mobile robot with a single active caster wheel configuration consists of at least two sets of active wheels. By substituting the parameters of these two sets of active

wheels into the term v_{cx} in equation (6), since the v_{cx} values calculated from both sets of steering wheels should be equal, we arrive at:

$$\begin{aligned} R_1 \dot{\theta}_1 \cos \alpha_1 + l_1 \omega \sin \varphi_1 + B_1 (\omega + \dot{\alpha}_1) \sin \alpha_1 = \\ R_2 \dot{\theta}_2 \cos \alpha_2 + l_2 \omega \sin \varphi_2 + B_2 (\omega + \dot{\alpha}_2) \sin \alpha_2 \end{aligned} \quad (10)$$

Bringing B to the left side of the equation yields:

$$\omega = \frac{B_2 \dot{\alpha}_2 \sin \alpha_2 - B_1 \dot{\alpha}_1 \sin \alpha_1 + R_2 \dot{\theta}_2 \cos \alpha_2 - R_1 \dot{\theta}_1 \cos \alpha_1}{B_1 \sin \alpha_1 - B_2 \sin \alpha_2 + L_1 \sin \varphi_1 - L_2 \sin \varphi_2} \quad (11)$$

The robot's angular velocity ω becomes known, substituting into equation (6) yields:

$$\begin{aligned} v_{cx} &= R_1 \dot{\theta}_1 c\alpha_1 + B_1 \dot{\alpha}_1 s\alpha_1 \\ &+ \left(\frac{B_2 \dot{\alpha}_2 s\alpha_2 - B_1 \dot{\alpha}_1 s\alpha_1 + R_2 \dot{\theta}_2 c\alpha_2 - R_1 \dot{\theta}_1 c\alpha_1}{B_1 s\alpha_1 - B_2 s\alpha_2 + L_1 s\varphi_1 - L_2 s\varphi_2} \right) (l_1 s\varphi_1 + B_1 s\alpha_1) \\ v_{cy} &= R_1 \dot{\theta}_1 s\alpha_1 - B_1 \dot{\alpha}_1 c\alpha_1 \\ &- \left(\frac{B_2 \dot{\alpha}_2 s\alpha_2 - B_1 \dot{\alpha}_1 s\alpha_1 + R_2 \dot{\theta}_2 c\alpha_2 - R_1 \dot{\theta}_1 c\alpha_1}{B_1 s\alpha_1 - B_2 s\alpha_2 + L_1 s\varphi_1 - L_2 s\varphi_2} \right) (l_1 s\varphi_1 + B_1 c\alpha_1) \end{aligned} \quad (12)$$

Where s and c are abbreviations for $\sin$ and $\cos$, respectively. The forward and inverse kinematics for the omni-directional mobile robot with steerable wheels have been fully solved. Using forward kinematics, the robot's odometry can be computed; a general algorithm for odometry is outlined as follows:

$$\begin{bmatrix} x_{t+1} \\ y_{t+1} \\ \theta_{t+1} \end{bmatrix} = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} + \begin{bmatrix} \cos \theta_t & -\sin \theta_t & 0 \\ \sin \theta_t & \cos \theta_t & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_{xt} \\ v_{yt} \\ \omega_t \end{bmatrix} dt \quad (13)$$

Where x_t, y_t, θ_t represents the position in the world coordinate system at time t , v_{xt}, v_{yt}, ω_t denotes the velocity in the robot's coordinate system at the same instant, and dt is the computation cycle. This section's content is embodied within the ROScontrol controller. Specifically, the velocities obtained from the inverse kinematics are transmitted to the robotHW for communication with the robot's servo motors, while the odometry calculated from forward kinematics contributes to the robot's localization process.

4 Motion Planning Based on Gaussian Processes

The motion planning based on GP is a versatile probabilistic framework employed to tackle challenges such as localization, path planning, and state estimation, particularly excelling in scenarios fraught with uncertainty, noise, and incomplete information. Its inherent capability to handle uncertainties renders it

highly effective in robotic navigation and state prediction tasks. To incorporate randomness and enhance the exploratory capacity of the trajectory planning algorithm, thereby endowing it with a superior ability to escape local optima, this paper adopts a Gaussian Process distribution as the prior for trajectory variables. The entire motion planning algorithm is bifurcated into two stages: a Gaussian Process-based path planning method followed by cubic spline trajectory planning.

4.1 Gaussian Process-based Path Planning

With the goal and initial points predefined, this section leverages a Gaussian random process to probabilistically generate multiple feasible trajectories connecting the starting point to the target point. Thereafter, the costs associated with these trajectories are meticulously evaluated by employing a cost map derived from obstacle data. The trajectory incurring the least cost is then selected as the pivotal trajectory guiding the subsequent trajectory generation phase. To ascertain potential obstacle infringements, the cost metric serves as an indicator, prompting an expansion of the search space upon detection of penetration and a constriction thereof when navigation remains collision-free. Initially, in the course of this study, a two-dimensional support vector $t_{\text{support}} = [0.0, 1.0]^T$ is introduced, which represents a reference point or datum in the context of the problem under consideration. To construct a model based on Gaussian Process Regression (GPR), an appropriate kernel function must be selected to characterize the inherent relationships among data points. Here, the Radial Basis Function (RBF) kernel, also known as the Gaussian kernel, is employed due to its smooth and globally differentiable properties. The expression for the RBF kernel is given by:

$$K_{\text{support}}(t_{\text{support}}, t'_{\text{support}}) = \sigma^2 \exp\left(-\frac{\|t_{\text{support}} - t'_{\text{support}}\|^2}{2\ell^2}\right) \quad (14)$$

In this formula, $\|t_{\text{support}} - t'_{\text{support}}\|^2$ represents the squared Euclidean distance between two vectors, σ is the variance parameter that controls the amplitude of function variations, and ℓ is the length scale parameter affecting the breadth of the function's spatial spread. This text first calculates the covariance $K_{\text{support}}(t_{\text{support}}, t_{\text{support}})$ between two identical support points, which serves as a fundamental component in constructing the Gaussian process model.

Subsequently, to ensure the positive definiteness and numerical stability of the covariance matrix, a Laplacian regularization term ϵI is added to the covariance matrix K_{support} computed from the support points, with ϵ being a very small positive real number and I representing the identity matrix. Thereafter, the Cholesky decomposition (LLT decomposition) is employed to solve the covariance matrix after regularization, yielding the lower triangular matrix L . This decomposition facilitates efficient computation of predictive distributions and associated statistics. After constructing the model, the focus shifts to the

data points to be predicted, namely, the set of test points t_{test} , which may be a set of discrete points generated through uniform interpolation from a continuous interval. For each test point, the covariance matrix $K_{support, test}$ is computed between this point and the support points; this matrix encapsulates the similarity between the test point and the unique set of support points.

$$K_{support, test}(t_{support}, t_{test}) = \sigma^2 \exp\left(-\frac{\|t_{support} - t_{test}\|^2}{2\ell^2}\right) \quad (15)$$

Then, leveraging the precomputed matrix L , this work addresses the covariance matrix between the test points and the support points, facilitating the derivation of the predictive distribution. In acknowledgment of the correlations among the test points, a posterior covariance matrix K_{test} is constructed.

$$K_{test}(t_{test}, t'_{test}) = \sigma^2 \exp\left(-\frac{\|t_{test} - t'_{test}\|^2}{2\ell^2}\right) \quad (16)$$

Building upon this, to accurately estimate the function values and their uncertainties at the test points, a modified approximation of the posterior covariance matrix in its lower triangular form, denoted as $L_{posterior}$, is established. This is accomplished by introducing yet another small positive regularization term ϵ' to K_{test} , while accounting for the effect of subtracting the diagonal elements of $K_{support, test}$ transformed by some operation L .

$$L_{posterior} = \text{chol}(K_{test} + \epsilon' I - K_{support, test}^\top L^{-\top} L^{-1} K_{support, test}) \quad (17)$$

Finally, with the posterior covariance matrix $L_{posterior}$ obtained through the above steps and its Cholesky decomposition (LLT), random paths are generated based on a Gaussian kernel and the mean trajectory. The mean trajectory is given by $\mu(x)$, where x represents an n -dimensional input variable, and each row corresponds to a data point. Meanwhile, $\mu(\mathbf{x}) \in \mathbb{R}^m$ denotes that the generated trajectory has m degrees of freedom; for instance, in the case of trajectory planning within a two-dimensional map space, $m = 2$.

Given a set of N discrete input points $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$, the mean path matrix of the corresponding Gaussian process is denoted as M , where $M_{ij} = \mu_j(\mathbf{x}_i)$. To generate P random path samples deviating from this mean path, a stochastic perturbation is defined. For each input point x_i , an m -dimensional vector of independent and identically distributed standard normal random variables $\epsilon_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is generated. For each sample p ($1 \leq p \leq P$) to be generated, the following procedure is carried out:

$$\mathbf{Y}_p = \mathbf{M} + \mathbf{L} \cdot \epsilon_p \quad (18)$$

All $\mathbf{Y}_p$ together form P random path samples, each of which conducts a random search based on the original path. Following this, the cost of each path is computed using a cost map. This work utilizes the `costmap_2d` component

within ROS to equip mobile robots with the capability to build and update two-dimensional cost maps, thereby facilitating a quantitative evaluation of potential paths by the path planning algorithm. Below, the process of calculating path costs using costmap_2d is elaborated upon.

Initially, costmap_2d constructs a gridded environmental model by fusing sensor data, such as that from lidars, depth cameras, etc. Each grid cell is assigned a cost value, which typically correlates with factors like obstacle proximity, terrain roughness, and dynamical constraints. The cost function can generally be expressed as:

$$C(x, y) = f(d_{\text{obs}}, d_{\text{inflated}}) \quad (19)$$

Wherein, (x, y) signifies the grid coordinates, d_{obs} denotes the measured distance from the current grid cell to the nearest obstacle, and d_{inflated} represents the additional inflation distance applied for safety, creating a buffer zone around obstacles. Within costmap_2d, obstacle costs are quantified using non-negative values, escalating as the proximity to obstacles increases, while remaining minimal or zero in unobstructed open spaces. Significantly, the inflation layer bolsters the costs of cells neighboring obstacles to assure a secure clearance for the robot. When evaluating the aggregate cost of a path composed of a series of grid points, a cumulative cost approach is employed:

$$\text{Cost}(T_i) = \sum_{i=1}^n w_i \cdot C(x_i, y_i) \quad (20)$$

In this context, $T_i = [(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)]$ represents the path under evaluation, a sequence comprising a series of adjacent grid points; $C(x_i, y_i)$ denotes the local cost at the i -th point along the path; w_i is a weighting factor that reflects the relative importance of different locations, inversely proportional to factors such as the distance from the start or goal points, thereby prioritizing path segments closer to crucial locations. Consequently, a set $\mathcal{T} = \{T_1, T_2, \dots, T_N\}$ of candidate trajectories is obtained, with each trajectory T_i associated with a cost function $\text{cost}(T_i)$, which gauges its safety, efficiency, or other pertinent criteria. The trajectory with the minimum cost is then selected as the mean path for the next planning iteration, mathematically represented as:

$$T_{\text{min-cost}} = \arg \min_{T_i \in \mathcal{T}} C(T_i) \quad (21)$$

As shown in fig.3, depending on the comparison between the minimum-cost trajectory and a pre-established collision threshold $C_{\text{threshold}}$, the parameters of the Gaussian kernel σ are adjusted dynamically. When $C(T_{\text{min-cost}}) > C_{\text{threshold}}$, indicating that the cost of the optimal trajectory is excessively high and potentially entails significant collision risks, the Gaussian kernel parameter σ is increased to broaden the search scope—this adjustment aims to explore a larger solution space, represented as $\sigma \leftarrow \alpha \cdot \sigma$, where $\alpha > 1$ is an expansion factor. Conversely, if the risk is assessed as lower, σ is decreased to narrow the search, implementing a strategy denoted as $\sigma \leftarrow \beta \cdot \sigma$, with $0 < \beta < 1$ functioning as a contraction factor.

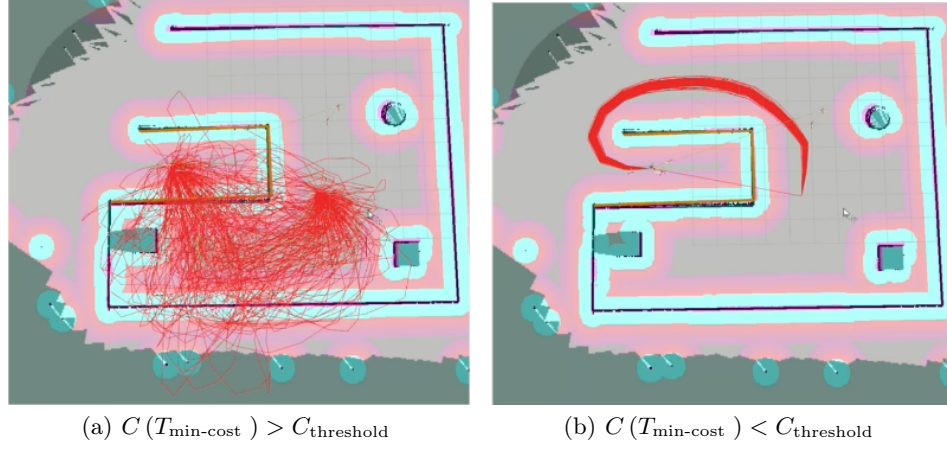


Fig. 3. The variation pattern of σ

4.2 Cubic Spline Interpolation

During the path planning stage, the algorithm employed performs a spatial search based on a finite number of key points and successfully generates a continuous path that avoids obstacles. However, this path is represented merely as a series of discrete spatial coordinate points, which are insufficient to accurately depict the requirement for smooth trajectories in complex environments. This section adopts cubic spline interpolation techniques to fit the existing set of discrete path points, thereby achieving a smoother trajectory. Let $S(t)$ represent a cubic spline curve, which is expressed as a piecewise cubic polynomial and possesses continuous second-order derivatives. At the node t_i , let the value of the second derivative be $S''(t_i) = M_i (i = 0, 1, \dots, n)$, and M_i denotes a specific parameter related to the position. According to [2], the cubic spline curve equation is derived as follows:

$$\begin{aligned}
 S(t) = & \frac{(t_i - t)^3}{6h_i} M_{i-1} + \frac{(t - t_{i-1})^3}{6h_i} M_i + \left(y_{i-1} - \frac{h_i^2}{6} M_{i-1} \right) \frac{t_i - t}{h_i} \\
 & + \left(y_i - \frac{h_i^2}{6} M_i \right) \frac{t - t_{i-1}}{h_i}, \quad t_{i-1} \leq t \leq t_i.
 \end{aligned} \tag{22}$$

Given that the initial and final velocities for the robot trajectory planning are known, the second type of boundary condition is employed, specified as: $y'_0 =$

$f'(a), y'_n = f'(b)$. Derive the three bending moment equations:

$$\begin{pmatrix} 2 & 1 & & & \\ \mu_1 & 2 & \lambda_1 & & \\ & \mu_2 & 2 & \lambda_2 & \\ & & \ddots & \ddots & \ddots \\ & & & \mu_{n-1} & 2 & \lambda_{n-1} \\ & & & & 1 & 2 \end{pmatrix} \begin{pmatrix} M_0 \\ M_1 \\ M_2 \\ \vdots \\ M_{n-1} \\ M_n \end{pmatrix} = \begin{pmatrix} d_0 \\ d_1 \\ d_2 \\ \vdots \\ d_{n-1} \\ d_n \end{pmatrix} \quad (23)$$

where:

$$\begin{aligned} \mu_i &= \frac{h_i}{h_i + h_{i+1}}, \quad \lambda_i = \frac{h_{i+1}}{h_i + h_{i+1}} = 1 - \mu_i \\ d_0 &= \frac{6}{h_1} \left(\frac{y_1 - y_0}{h_1} - y'_0 \right) = 6f[x_0, x_0, x_1], \\ d_n &= \frac{6}{h_n} \left(y'_n - \frac{y_n - y_{n-1}}{h_n} \right) = 6f[x_{n-1}, x_n, x_n], \\ d_i &= \frac{6}{h_i + h_{i+1}} \left[\frac{y_{i+1} - y_i}{h_{i+1}} - \frac{y_i - y_{i-1}}{h_i} \right] = 6f[x_{i-1}, x_i, x_{i+1}]. \end{aligned} \quad (24)$$

Where $i = 1, 2, \dots, n-1$, parameter h_i needs to be determined, and in this work, it is computed by adopting an average velocity of v_m . For any arbitrary pair of consecutive coordinate points y_{i-1}, y_i , along the prescribed path, the pertinent formula is expressed as follows:

$$h_i = \frac{\|y_i - y_{i-1}\|_2}{v_m} \quad (25)$$

Upon completion of the trajectory planning, the generated trajectory is input into a PID controller, which computes the current command velocity for the robot. This command is then transmitted to the lower-level controller, thereby accomplishing the control task.

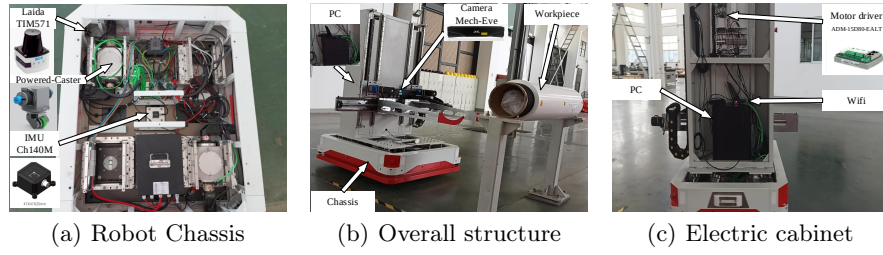
5 Experiments and Results

5.1 Experimental Hardware and Platform Construction

The hardware platform of the Powered-Caster holonomic robot used in this experiment is shown in Fig.4, which consists of Two-Powered-Caster chassis, two TIM-571 LIDARs, Ch140M IMU, Mech-Eye camera and PC, where the CPU of the PC is i5-13600KF with 16 GB memory and RTX4060Ti GPU. The data processing side is the operating system of Ubuntu 20.04.

5.2 Experiment of Workpiece Handling

This thesis utilizes a scenario involving the handling of cylindrical workpieces as its experimental context, as depicted in fig.5. It outlines a process where a robot

**Fig. 4.** Hardware Platform

commences from an initial point, traverses an environment replete with obstacles, ultimately arriving at the workpiece's location. Furthermore, the study contrasts the efficacy of three distinct methods: A* coupled with TEB (Topology-based Elastic Band), A* combined with DWA (Dynamic Window Approach), and the novel methodology proposed within this paper.

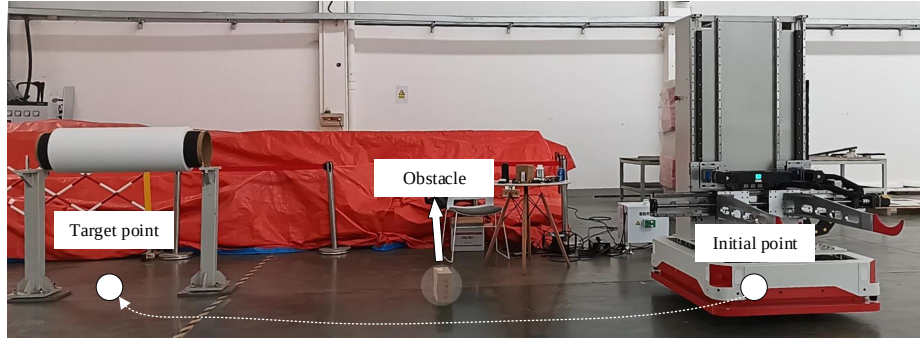
**Fig. 5.** Experiment

Fig.6 compares the sequences of velocity commands produced by three planning algorithms. Subplots a, b, and c respectively depict the velocities in the x-direction, y-direction, and the angular velocity around the z-axis in the robot's coordinate system. Here, the red line represents the performance of A*+DWA, the green line shows the outcome of A*+TEB, while the blue line signifies our proposed method. Our approach achieves the target destination first, with a time of 14.253 seconds. The DWA and TEB methods, in comparison, take 14.651 seconds and 17.396 seconds, respectively. In this experiment, to ensure stability, we set the frequency of our proposed method to 50Hz, whereas the other two methods could only operate at a maximum frequency of 30Hz. If required, our planning algorithm has the capability to reach up to 1kHz to guarantee real-time performance.

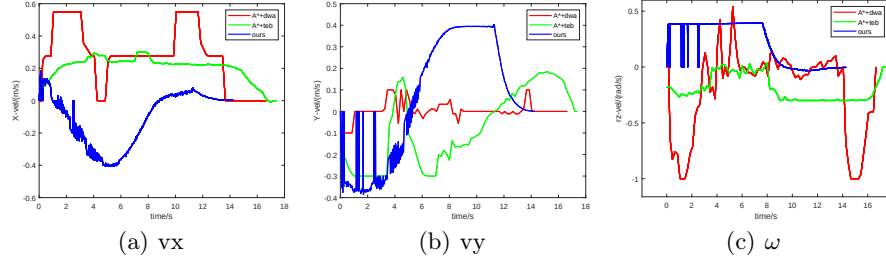


Fig. 6. The sequence of velocity commands generated by three planning algorithms

During the first three seconds, our algorithm generates plans with moments of zero velocity, a phenomenon arising from the utilization of the `costmap_2d` package for generating obstacle cost maps. The planning process involves frequent queries for cost values to assess potential paths. Occasionally, under high query loads, the `costmap_2d` momentarily returns a zero cost for occupied areas, misleading our algorithm into perceiving a straight-line path as the most optimal due to its perceived minimal distance. This misinterpretation narrows the search space for subsequent path planning, focusing exclusively on direct trajectories.

However, when the `costmap_2d` subsequently functions correctly and accurately reports the costs of these previously misread regions, all paths explored based on the linear assumption and the narrowed search radius are revealed to incur severe collisions. To ensure safety, this detection triggers a safeguard mechanism within our algorithm, promptly reducing the robot's velocity to zero to avert any potential accidents.

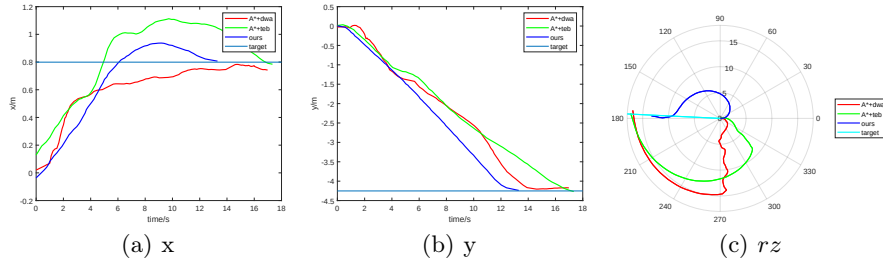


Fig. 7. Pose-time curves

Fig.7 illustrates the pose-time curves for the three methodologies, demonstrating that the DWA algorithm maintains a relatively constant heading during the initial half of the traversal, only initiating significant attitude adjustments as it approaches the target, which leads to deviations from the target and an increase in positional errors during the maneuvering phase. Overall, it is evident

that the trajectory smoothness produced by our method significantly surpasses that of TEB and DWA. This superiority is attributed to our approach’s continuous global path planning, in contrast to the other two algorithms that plan within limited windows only.

Table 1. The positional error after the robot halts relative to the target point.

Algorithm	x error (mm)	y error (mm)	rz error (°)
A*+DWA	57.2	80.1	2.5
A*+TEB	14.9	15.7	0.44
Ours	7.2	13.5	0.66

As illustrated in Table 1, ours demonstrates superior performance in terms of positional errors along both the x and y axes compared to the other two methods. Furthermore, akin to TEB, ours also achieves an angular error of less than 1 degree, showcasing its commendable precision in orientation control. We have uploaded the experimental video and code to the internet.

6 Conclusions

This paper integrates Gaussian Process-based path planning with cubic spline interpolation to introduce a novel Gaussian Process-driven trajectory planning approach, which is then implemented on a manipulator robot for object handling tasks. Empirical results affirm that our proposed methodology, when applied to an actively steered holonomic mobile robot, surpasses conventional algorithms, particularly in aspects of positional tracking accuracy, motion smoothness, and computational frequency. Notably, it achieves a significantly higher computational rate, with the upper limit reaching approximately 1kHz, far exceeding those of traditional methods. Additionally, our study has unveiled that the performance of our proposed scheme is constrained by the limitations of the costmap_2d package. Consequently, future work will entail replacing the costmap_2d component to enhance the overall efficiency and effectiveness of the algorithm. We have uploaded the experimental video³ and code⁴ to the internet.

Acknowledgements. This work is supported by the National Natural Science Foundation of China (Grant No. 52175029).

References

1. Latombe, J.-C.: Introduction and Overview. In: Latombe, J.-C. (ed.) Robot Motion Planning. pp. 1–57. Springer US, Boston, MA (1991)

³ <https://www.bilibili.com/video/BV1jD421G7h5>

⁴ https://github.com/yeying256/agv_housheng

2. Gautschi W.: Approximation and Interpolation. In: Gautschi W (ed) Numerical Analysis. Birkhäuser, Boston, pp 55–158 (2012)
3. Fox, D., Burgard, W., Thrun, S.: Controlling synchro-drive robots with the dynamic window approach to collision avoidance. In: Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS '96. pp. 1280–1287 vol.3 (1996)
4. Quinlan, S., Khatib, O.: Elastic bands: connecting path planning and control. In: [1993] Proceedings IEEE International Conference on Robotics and Automation. pp. 802–807 vol.2 (1993)
5. Rösmann, C., Hoffmann, F., Bertram, T.: Kinodynamic trajectory optimization and control for car-like robots. In: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). pp. 5681–5686 (2017)
6. Zai, W., Lin, Q., Wang, S., Lin, X.: Path Planning for Wheeled Robots Based on the Fusion of Improved A* and TEB Algorithms. In: 2023 China Automation Congress (CAC). pp. 3257–3261 (2023)
7. Dobrevski, M., Skočaj, D.: Dynamic Adaptive Dynamic Window Approach. IEEE Transactions on Robotics. 1–14 (2024). <https://doi.org/10.1109/TRO.2024.3400932>
8. Stanford Artificial Intelligence Laboratory et al., “Robotic operating system.”, <https://www.ros.org>
9. Zheng, H., Dai, M., Zhang, Z., Xia, Z., Zhang, G., Jia, F.: The Navigation Based on Hybrid A Star and TEB Algorithm Implemented in Obstacles Avoidance. In: 2023 29th International Conference on Mechatronics and Machine Vision in Practice (M2VIP). pp. 1–6 (2023)
10. Jiang, D., Du, L., Li, S., Wang, M., Zhang, H., Chen, X., Sun, Y.: An Improved Dynamic Window Approach Based on Reinforcement Learning for the Trajectory Planning of Automated Guided Vehicles. IEEE Access. 12, 36016–36025 (2024). <https://doi.org/10.1109/ACCESS.2024.3373446>
11. Imamoglu, M.R., Sumer, E., Temeltas, H.: A Comparison of Local Planner Algorithms for a ROS-based Omnidirectional Mobile Robot. In: 2023 8th International Conference on Robotics and Automation Engineering (ICRAE). pp. 26–30 (2023)
12. Liu, B., Jiang, G., Zhao, F., Mei, X.: Collision-Free Motion Generation Based on Stochastic Optimization and Composite Signed Distance Field Networks of Articulated Robot. IEEE Robotics and Automation Letters. 8, 7082–7089 (2023). <https://doi.org/10.1109/LRA.2023.3311357>
13. Jia, W., Yang, G., Gu, L., Zheng, T.: Dynamics modelling of a mobile manipulator with powered castor wheels. In: 2017 IEEE International Conference on Cybernetics and Intelligent Systems (CIS) and IEEE Conference on Robotics, Automation and Mechatronics (RAM). pp. 730–735 (2017)
14. Jia, W., Yang, G., Zhang, C., Chen, G.: Dynamic modeling with nonsqueezing load distribution for omnidirectional mobile robots with powered caster wheels. In: 2018 13th IEEE Conference on Industrial Electronics and Applications (ICIEA). pp. 2327–2332 (2018)